

Planar data classification with one hidden layer

In this assignment, students will build first neural network having a hidden layer. You will see a big difference between this model and the one you implemented using logistic regression.

You will learn how to:

- Implement a 2-class classification neural network with a single hidden layer
- Use units with a non-linear activation function, such as tanh
- Compute the cross entropy loss
- Implement forward and backward propagation

1 - Packages

Let's first import all the packages that you will need during this assignment.

- [numpy](https://www.numpy.org/) (<https://www.numpy.org/>) is the fundamental package for scientific computing with Python.
- [sklearn](http://scikit-learn.org/stable/) (<http://scikit-learn.org/stable/>) provides simple and efficient tools for data mining and data analysis.
- [matplotlib](http://matplotlib.org) (<http://matplotlib.org>) is a library for plotting graphs in Python.
- testCases provides some test examples to assess the correctness of your functions
- planar_utils provide various useful functions used in this assignment

```
In [ ]: # Package imports
import numpy as np
import matplotlib.pyplot as plt
from testCases_v2 import *
import sklearn
import sklearn.datasets
import sklearn.linear_model
from planar_utils import plot_decision_boundary, sigmoid, load_planar_dataset,
%matplotlib inline
```

2 - Dataset

First, let's get the dataset you will work on. The following code will load a "flower" 2-class dataset into variables X and Y . Visualize the dataset using matplotlib. The data looks like a "flower" with some red (label $y=0$) and some blue ($y=1$) points. Your goal is to build a model to fit this data. In other words, we want the classifier to define regions as either red or blue.

```
In [ ]: X, Y = load_planar_dataset()
# Visualize the data:
plt.scatter(X[0, :], X[1, :], c=Y[0, :], s=40, cmap=plt.cm.Spectral); # modified
```

You have:

- a numpy-array (matrix) X that contains your features (x1, x2)
- a numpy-array (vector) Y that contains your labels (red:0, blue:1).

Lets first get a better sense of what our data is like.

Exercise: How many training examples do you have? In addition, what is the shape of the variables X and Y ?

Hint: How do you get the shape of a numpy array? ([help](https://docs.scipy.org/doc/numpy/reference/generated/numpy.ndarray.shape.html)) (<https://docs.scipy.org/doc/numpy/reference/generated/numpy.ndarray.shape.html>)

```
In [ ]: ### START CODE HERE ### (~ 3 lines of code)

### END CODE HERE ###

print ('The shape of X is: ' + str(shape_X))
print ('The shape of Y is: ' + str(shape_Y))
```

Expected Output:

```
shape of X (2, 400)
shape of Y (1, 400)
m          400
```

3 - Simple Logistic Regression

Before building a full neural network, lets first see how logistic regression performs on this problem. You can use sklearn's built-in functions to do that. Run the code below to train a logistic regression classifier on the dataset.

```
In [ ]: # Train the logistic regression classifier
clf = sklearn.linear_model.LogisticRegressionCV();
clf.fit(X.T, Y.T);

# Plot the decision boundary for logistic regression
plot_decision_boundary(lambda x: clf.predict(x), X, Y)
plt.title("Logistic Regression")

# Print accuracy
LR_predictions = clf.predict(X.T)
print('Accuracy of logistic regression: %d ' % float((np.dot(Y,LR_predictions)
'% ' + "(percentage of correctly labelled datapoints)"))
```

Expected Output:

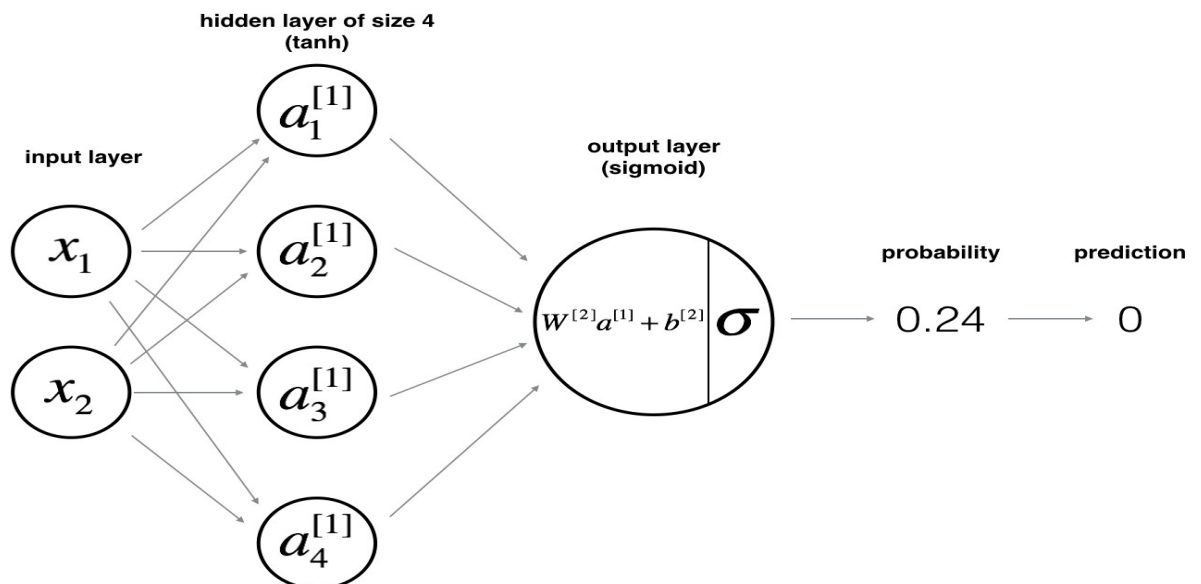
Accuracy 47%

Interpretation: The dataset is not linearly separable, so logistic regression doesn't perform well. Hopefully a neural network will do better. Let's try this now!

4 - Neural Network model

Logistic regression did not work well on the "flower dataset". You are going to train a Neural Network with a single hidden layer.

Here is our model:



Mathematically:

For one example $x^{(i)}$: $z^{[1]}(i) = W^{[1]} x^{(i)} + b^{[1]}$ $a^{[1]}(i) = \tanh(z^{[1]}(i))$ $z^{[2]}(i) = W^{[2]} a^{[1]}(i) + b^{[2]}$ $\hat{y}^{(i)} = a^{[2]}(i) = \sigma(z^{[2]}(i))$ $y^{(i)}_{\text{prediction}} = \begin{cases} 1 & \text{if } a^{[2]}(i) > 0.5 \\ 0 & \text{otherwise} \end{cases}$

Given the predictions on all the examples, you can also compute the cost J as follows: $J = -\frac{1}{m} \sum_{i=0}^m \left(y^{(i)} \log(a^{[2] (i)}) + (1 - y^{(i)}) \log(1 - a^{[2] (i)}) \right)$

Reminder: The general methodology to build a Neural Network is to:

1. Define the neural network structure (# of input units, # of hidden units, etc).
2. Initialize the model's parameters
3. Loop:
 - Implement forward propagation
 - Compute loss
 - Implement backward propagation to get the gradients
 - Update parameters (gradient descent)

You often build helper functions to compute steps 1-3 and then merge them into one function we call `nn_model()`. Once you've built `nn_model()` and learnt the right parameters, you can make predictions on new data.

4.1 - Defining the neural network structure

Exercise: Define three variables:

- `n_x`: the size of the input layer
- `n_h`: the size of the hidden layer (set this to 4)
- `n_y`: the size of the output layer

Hint: Use shapes of `X` and `Y` to find `n_x` and `n_y`. Also, hard code the hidden layer size to be 4.

In []:

```
def layer_sizes(X, Y):
    """
    Arguments:
    X -- input dataset of shape (input size, number of examples)
    Y -- labels of shape (output size, number of examples)

    Returns:
    n_x -- the size of the input layer
    n_h -- the size of the hidden layer
    n_y -- the size of the output layer
    """
    ### START CODE HERE ### (≈ 3 lines of code)

    ### END CODE HERE ###
```

```
In [ ]: X_assess, Y_assess = layer_sizes_test_case()
        (n_x, n_h, n_y) = layer_sizes(X_assess, Y_assess)
        print("The size of the input layer is: n_x = " + str(n_x))
        print("The size of the hidden layer is: n_h = " + str(n_h))
        print("The size of the output layer is: n_y = " + str(n_y))
```

Expected Output (these are not the sizes you will use for your network, they are just used to assess the function you've just coded).

n_x	5
n_h	4
n_y	2

4.2 - Initialize the model's parameters

Exercise: Implement the function `initialize_parameters()` .

Instructions:

- Make sure your parameters' sizes are right. Refer to the neural network figure above if needed.
- You will initialize the weights matrices with random values.
 - Use: `np.random.randn(a,b) * 0.01` to randomly initialize a matrix of shape (a,b).
- You will initialize the bias vectors as zeros.
 - Use: `np.zeros((a,b))` to initialize a matrix of shape (a,b) with zeros.

```
In [ ]: def initialize_parameters(n_x, n_h, n_y):
        """
        Argument:
        n_x -- size of the input layer
        n_h -- size of the hidden layer
        n_y -- size of the output layer

        Returns:
        params -- python dictionary containing your parameters:
                    W1 -- weight matrix of shape (n_h, n_x)
                    b1 -- bias vector of shape (n_h, 1)
                    W2 -- weight matrix of shape (n_y, n_h)
                    b2 -- bias vector of shape (n_y, 1)
        """

        np.random.seed(2) # we set up a seed so that your output matches ours altho

        ### START CODE HERE ### (= 4 lines of code)


        ### END CODE HERE ###

        assert (W1.shape == (n_h, n_x))
        assert (b1.shape == (n_h, 1))
        assert (W2.shape == (n_y, n_h))
        assert (b2.shape == (n_y, 1))

        parameters = {"W1": W1,
                      "b1": b1,
                      "W2": W2,
                      "b2": b2}
```

```
In [ ]: n_x, n_h, n_y = initialize_parameters_test_case()

        parameters = initialize_parameters(n_x, n_h, n_y)
        print("W1 = " + str(parameters["W1"]))
        print("b1 = " + str(parameters["b1"]))
        print("W2 = " + str(parameters["W2"]))
        print("b2 = " + str(parameters["b2"]))
```

Expected Output:

W1	[[-0.00416758 -0.00056267] [-0.02136196 0.01640271] [-0.01793436 -0.00841747] [0.00502881 -0.01245288]]
b1	[[0.] [0.] [0.] [0.]]
W2	[[-0.01057952 -0.00909008 0.00551454 0.02292208]]
b2	[[0.]]

4.3 - The Loop

Question: Implement `forward_propagation()` .

Instructions:

- Look above at the mathematical representation of your classifier.
- You can use the function `sigmoid()` . It is built-in (imported) in the notebook.
- You can use the function `np.tanh()` . It is part of the numpy library.
- The steps you have to implement are:
 1. Retrieve each parameter from the dictionary "parameters" (which is the output of `initialize_parameters()`) by using `parameters[".."]` .
 2. Implement Forward Propagation. Compute $Z^{[1]}$, $A^{[1]}$, $Z^{[2]}$ and $A^{[2]}$ (the vector of all your predictions on all the examples in the training set).
- Values needed in the backpropagation are stored in " cache ". The `cache` will be given as an input to the backpropagation function.

```
In [ ]: def forward_propagation(X, parameters):
        """
        Argument:
        X -- input data of size (n_x, m)
        parameters -- python dictionary containing your parameters (output of initialization)

        Returns:
        A2 -- The sigmoid output of the second activation
        cache -- a dictionary containing "Z1", "A1", "Z2" and "A2"
        """
        # Retrieve each parameter from the dictionary "parameters"
        # W1, b1, W2, b2
        ### START CODE HERE ### (~ 4 lines of code)

        ### END CODE HERE ###

        # Implement Forward Propagation to calculate A2 (probabilities)
        ### START CODE HERE ### (~ 4 lines of code)

        ### END CODE HERE ###

        assert(A2.shape == (1, X.shape[1]))

        # Values needed in the backpropagation are stored in "cache". This will be
        cache = {"Z1": Z1,
                "A1": A1,
                "Z2": Z2,
                "A2": A2}
```

```
In [ ]: X_assess, parameters = forward_propagation_test_case()
        A2, cache = forward_propagation(X_assess, parameters)

        # Note: we use the mean here just to make sure that your output matches ours.
```

Expected Output:

```
0.262818640198 0.091999045227 -1.30766601287
0.212877681719
```

Now that you have computed $A^{[2]}$ (in the Python variable "A2"), which contains $a^{[2](i)}$ for every example, you can compute the cost function as follows:

$$J = -\frac{1}{m} \sum_{i=1}^m \left(y^{(i)} \log(a^{[2](i)}) + (1 - y^{(i)}) \log(1 - a^{[2](i)}) \right)$$

Exercise: Implement `compute_cost()` to compute the value of the cost J .

Instructions:

- There are many ways to implement the cross-entropy loss. To help you, we give you how we would have implemented $-\sum_{i=0}^m y^{(i)} \log(a^{[2]}(i))$:

```
logprobs = np.multiply(np.log(A2),Y)
cost = - np.sum(logprobs)           # no need to use a for loop!
```

(you can use either `np.multiply()` and then `np.sum()` or directly `np.dot()`). Note that if you use `np.multiply` followed by `np.sum` the end result will be a type `float` , whereas if you use `np.dot` , the result will be a 2D numpy array. We can use `np.squeeze()` to remove redundant dimensions (in the case of single float, this will be reduced to a zero-dimension array). We can cast the array as a type `float` using `float()` .

```
In [ ]: def compute_cost(A2, Y, parameters):
        """
        Computes the cross-entropy cost given in equation (13)

        Arguments:
        A2 -- The sigmoid output of the second activation, of shape (1, number of examples)
        Y -- "true" labels vector of shape (1, number of examples)
        parameters -- python dictionary containing your parameters W1, b1, W2 and b2
        [Note that the parameters argument is not used in this function,
        but the auto-grader currently expects this parameter.
        Future version of this notebook will fix both the notebook
        and the auto-grader so that `parameters` is not needed.
        For now, please include `parameters` in the function signature,
        and also when invoking this function.]

        Returns:
        cost -- cross-entropy cost given equation (13)

        """

        m = Y.shape[1] # number of examples

        # Compute the cross-entropy cost
        ### START CODE HERE ### (= 2 lines of code)

        ### END CODE HERE ###

        cost = float(np.squeeze(cost)) # makes sure cost is the dimension we expect
                                       # E.g., turns [[17]] into 17
        assert(isinstance(cost, float))
```

```
In [ ]: A2, Y_assess, parameters = compute_cost_test_case()
```

Expected Output:

cost 0.693058761...

Using the cache computed during forward propagation, you can now implement backward propagation.

Question: Implement the function `backward_propagation()` .

Instructions: Backpropagation is usually the hardest (most mathematical) part in deep learning. To help you, here again is the slide from the lecture on backpropagation. You'll want to use the six equations on the right of this slide, since you are building a vectorized implementation.

Summary of gradient descent

$$dz^{[2]} = a^{[2]} - y$$

$$dW^{[2]} = dz^{[2]} a^{[1]T}$$

$$db^{[2]} = dz^{[2]}$$

$$dz^{[1]} = W^{[2]T} dz^{[2]} * g^{[1]'}(z^{[1]})$$

$$dW^{[1]} = dz^{[1]} x^T$$

$$db^{[1]} = dz^{[1]}$$

$$dZ^{[2]} = A^{[2]} - Y$$

$$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1, keepdims = True)$$

$$dZ^{[1]} = W^{[2]T} dZ^{[2]} * g^{[1]'}(Z^{[1]})$$

$$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$$

$$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$$

Andrew Ng

- Tips:
 - To compute $dZ^{[1]}$ you'll need to compute $g^{[1]'}(Z^{[1]})$. Since $g^{[1]}(\cdot)$ is the tanh activation function, if $a = g^{[1]}(z)$ then $g^{[1]'}(z) = 1 - a^2$. So you can compute $g^{[1]'}(Z^{[1]})$ using `(1 - np.power(A1, 2))` .

```

In [ ]: def backward_propagation(parameters, cache, X, Y):
        """
        Implement the backward propagation using the instructions above.

        Arguments:
        parameters -- python dictionary containing our parameters
        cache -- a dictionary containing "Z1", "A1", "Z2" and "A2".
        X -- input data of shape (2, number of examples)
        Y -- "true" labels vector of shape (1, number of examples)

        Returns:
        grads -- python dictionary containing your gradients with respect to different parameters
        """
        m = X.shape[1]

        # First, retrieve W1 and W2 from the dictionary "parameters".
        ### START CODE HERE ### (= 2 lines of code)

        ### END CODE HERE ###

        # Retrieve also A1 and A2 from dictionary "cache".
        ### START CODE HERE ### (= 2 lines of code)

        ### END CODE HERE ###

        # Backward propagation: calculate dw1, db1, dw2, db2.
        ### START CODE HERE ### (= 6 lines of code, corresponding to 6 equations on the notebook)

        ### END CODE HERE ###

        grads = {"dw1": dw1,
                  "db1": db1,
                  "dw2": dw2,
                  "db2": db2}

```

```

In [ ]: parameters, cache, X_assess, Y_assess = backward_propagation_test_case()

        grads = backward_propagation(parameters, cache, X_assess, Y_assess)
        print ("dw1 = " + str(grads["dw1"]))
        print ("db1 = " + str(grads["db1"]))
        print ("dw2 = " + str(grads["dw2"]))
        print ("db2 = " + str(grads["db2"]))

```

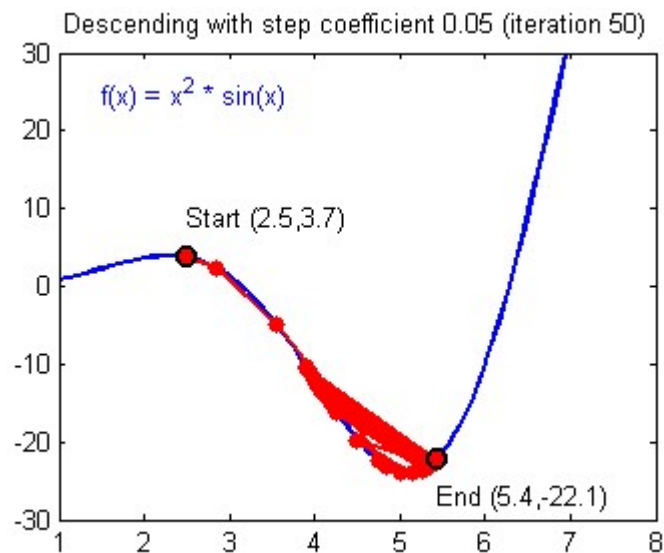
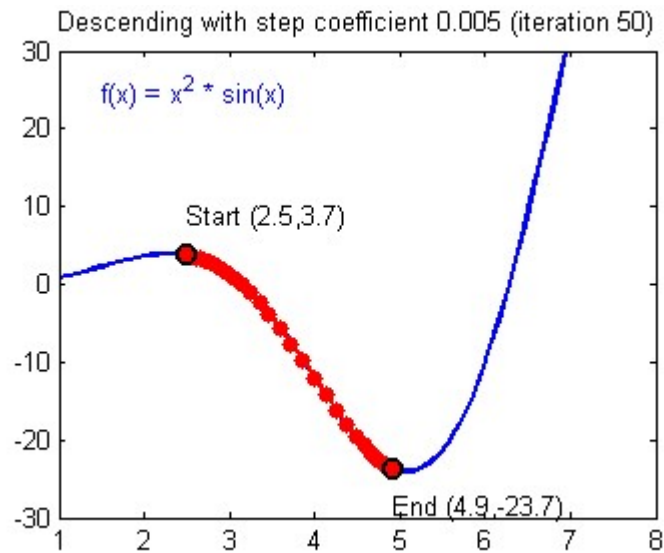
Expected output:

dW1	[[0.00301023 -0.00747267] [0.00257968 -0.00641288] [-0.00156892 0.003893] [-0.00652037 0.01618243]]
db1	[[0.00176201] [0.00150995] [-0.00091736] [-0.00381422]]
dW2	[[0.00078841 0.01765429 -0.00084166 -0.01022527]]
db2	[[-0.16655712]]

Question: Implement the update rule. Use gradient descent. You have to use (dW1, db1, dW2, db2) in order to update (W1, b1, W2, b2).

General gradient descent rule: $\theta = \theta - \alpha \frac{\partial J}{\partial \theta}$ where α is the learning rate and θ represents a parameter.

Illustration: The gradient descent algorithm with a good learning rate (converging) and a bad learning rate (diverging). Images courtesy of Adam Harley.



```
In [ ]: def update_parameters(parameters, grads, learning_rate):
        """
        Updates parameters using the gradient descent update rule given above

        Arguments:
        parameters -- python dictionary containing your parameters
        grads -- python dictionary containing your gradients

        Returns:
        parameters -- python dictionary containing your updated parameters
        """
        # Retrieve each parameter from the dictionary "parameters"
        ### START CODE HERE ### (= 4 lines of code)

        ### END CODE HERE ###

        # Retrieve each gradient from the dictionary "grads"
        ### START CODE HERE ### (= 4 lines of code)

        ## END CODE HERE ###

        # Update rule for each parameter
        ### START CODE HERE ### (= 4 lines of code)

        ### END CODE HERE ###

        parameters = {"W1": W1,
                       "b1": b1,
                       "W2": W2,
                       "b2": b2}
```

```
In [ ]: parameters, grads = update_parameters_test_case()
        parameters = update_parameters(parameters, grads, 1.2)

        print("W1 = " + str(parameters["W1"]))
        print("b1 = " + str(parameters["b1"]))
        print("W2 = " + str(parameters["W2"]))
        print("b2 = " + str(parameters["b2"]))
```

Expected Output:

```

W1 [[-0.00643025 0.01936718] [-0.02410458
    0.03978052] [-0.01653973 -0.02096177] [
    0.01046864 -0.05990141]]

b1 [[ -1.02420756e-06] [ 1.27373948e-05] [
    8.32996807e-07] [-3.20136836e-06]]

W2 [[-0.01041081 -0.04463285 0.01758031
    0.04747113]]

b2 [[ 0.00010457]]

```

4.4 - Integrate parts 4.1, 4.2 and 4.3 in nn_model()

Question: Build your neural network model in `nn_model()` .

Instructions: The neural network model has to use the previous functions in the right order.

```

In [ ]: # NN_model
def nn_model(X, Y, n_h, learning_rate, num_iterations = 10000, print_cost=False):
    n_x = layer_sizes(X, Y)[0]
    n_y = layer_sizes(X, Y)[2]

    # Initialize parameters
    parameters = initialize_parameters(n_x, n_h, n_y)
    W1 = parameters["W1"]
    b1 = parameters["b1"]
    W2 = parameters["W2"]
    b2 = parameters["b2"]

    # Loop (gradient descent)
    for i in range(0, num_iterations):
        # Forward propagation. Inputs: "X, parameters". Outputs: "A2, cache"
        A2, cache = forward_propagation(X, parameters)
        # Cost function. Inputs: "A2, Y, parameters". Outputs: "cost"
        cost = compute_cost(A2, Y, parameters)
        # Backpropagation. Inputs: "parameters, cache, X, Y". Outputs: "grads"
        grads = backward_propagation(parameters, cache, X, Y)
        # Update rule for each parameter
        parameters = update_parameters(parameters, grads, learning_rate)
        # If print_cost=True, Print the cost every 1000 iterations
        if print_cost and i % 1000 == 0:
            print ("Cost after iteration %i: %f" %(i, cost))
    # Returns parameters Learnt by the model. They can then be used to predict
    return parameters

```

```

In [ ]: X_assess, Y_assess = nn_model_test_case()
parameters = nn_model(X_assess, Y_assess, 4, 1.02, num_iterations=10000, print_c
print("W1 = " + str(parameters["W1"]))
print("b1 = " + str(parameters["b1"]))
print("W2 = " + str(parameters["W2"]))

```

Expected Output:

	cost after iteration 0	0.692739
	$\vdots$	$\vdots$
W1	$\begin{bmatrix} -0.65848169 & 1.21866811 \\ 1.39377573 & 0.5792005 \end{bmatrix}$ $\begin{bmatrix} -0.76204273 \\ -1.10397703 \end{bmatrix}$ $\begin{bmatrix} 0.76773391 & -1.41477129 \end{bmatrix}$	
b1	$\begin{bmatrix} 0.287592 \\ 0.3511264 \end{bmatrix}$ $\begin{bmatrix} -0.2431246 \\ -0.35772805 \end{bmatrix}$	
W2	$\begin{bmatrix} -2.45566237 & -3.27042274 \\ 2.00784958 & 3.36773273 \end{bmatrix}$	
b2	$\begin{bmatrix} 0.20459656 \end{bmatrix}$	

4.5 Predictions

Question: Use your model to predict by building predict(). Use forward propagation to predict results.

Reminder: $y_{\text{prediction}} = \mathbb{1}_{\{\text{activation} > 0.5\}} = \begin{cases} 1 & \text{if activation} > 0.5 \\ 0 & \text{otherwise} \end{cases}$

As an example, if you would like to set the entries of a matrix X to 0 and 1 based on a threshold you would do: $X_{\text{new}} = (X > \text{threshold})$

```
In [ ]: def predict(parameters, X):
        """
        Using the learned parameters, predicts a class for each example in X

        Arguments:
        parameters -- python dictionary containing your parameters
        X -- input data of size (n_x, m)

        Returns
        predictions -- vector of predictions of our model (red: 0 / blue: 1)
        """

        # Computes probabilities using forward propagation, and classifies to 0/1 using a threshold
        ### START CODE HERE ### (= 2 lines of code)

        ### END CODE HERE ###
```

```
In [ ]: parameters, X_assess = predict_test_case()

        predictions = predict(parameters, X_assess)
```

Expected Output:

predictions mean 0.666666666667

4.6 Train and predict on a planar dataset.

It is time to run the model and see how it performs on a planar dataset. Run the following code to test your model with a single hidden layer of n_h hidden units.

```
In [ ]: # Build a model with a  $n_h$ -dimensional hidden layer
parameters = nn_model(X, Y, 4, 1.2, num_iterations = 10000, print_cost=True)

# Plot the decision boundary
plot_decision_boundary(lambda x: predict(parameters, x.T), X, Y)
plt.title("Decision Boundary for hidden layer size " + str(4))
# Print accuracy
predictions = predict(parameters, X)
```

Expected Output:

Cost after iteration
9000 0.218607

Accuracy90%

Accuracy is really high compared to Logistic Regression. The model has learnt the leaf patterns of the flower! Neural networks are able to learn even highly non-linear decision boundaries, unlike logistic regression.

Now, let's try out several hidden layer sizes.

4.6 - Tuning hidden layer size (a planar dataset)

Run the following code. It may take 1-2 minutes. You will observe different behaviors of the model for various hidden layer sizes.

```
In [ ]: # This may take about 2 minutes to run

plt.figure(figsize=(16, 32))
hidden_layer_sizes = [1, 2, 3, 4, 5, 20, 50]
for i, n_h in enumerate(hidden_layer_sizes):
    plt.subplot(5, 2, i+1)
    plt.title('Hidden Layer of size %d' % n_h)
    parameters = nn_model(X, Y, n_h, 1.2, num_iterations = 5000)
    plot_decision_boundary(lambda x: predict(parameters, x.T), X, Y)
    predictions = predict(parameters, X)
    accuracy = float((np.dot(Y, predictions.T) + np.dot(1-Y, 1-predictions.T))/fl
```

Interpretation:

- The larger models (with more hidden units) are able to fit the training set better, until eventually the largest models overfit the data.
- The best hidden layer size seems to be around $n_h = 5$. Indeed, a value around here seems to fit the data well without also incurring noticeable overfitting.
- You will also learn later about regularization, which lets you use very large models (such as $n_h = 50$) without much overfitting.

Optional questions:

Some questions that you can explore:

- What happens when you change the tanh activation for a sigmoid activation or a ReLU activation?
- Play with the learning_rate. What happens?
- What if we change the dataset? (See part 5 below!)

You've learnt to:

- Build a complete neural network with a hidden layer
- Make a good use of a non-linear unit
- Implemented forward propagation and backpropagation, and trained a neural network
- See the impact of varying the hidden layer size, including overfitting.

Nice work!

5) Performance on other datasets

If you want, you can rerun the whole notebook (minus the dataset part) for each of the following datasets.

```

In [ ]: # Datasets
noisy_circles, noisy_moons, blobs, gaussian_quantiles, no_structure = load_ext

datasets = {"noisy_circles": noisy_circles,
            "noisy_moons": noisy_moons,
            "blobs": blobs,
            "gaussian_quantiles": gaussian_quantiles}

### choose your dataset (Student change name to choose other datasets.)
dataset = "noisy_moons"

X, Y = datasets[dataset]
X, Y = X.T, Y.reshape(1, Y.shape[0])

# make blobs binary
if dataset == "blobs":
    Y = Y%2

# Visualize the data
plt.scatter(X[0, :], X[1, :], c=Y[0, :], s=40, cmap=plt.cm.Spectral);

```

Reference:

- <http://scs.ryerson.ca/~aharley/neural-networks/> (<http://scs.ryerson.ca/~aharley/neural-networks/>)
- <http://cs231n.github.io/neural-networks-case-study/> (<http://cs231n.github.io/neural-networks-case-study/>)